ONEINSURE | INTERNAL TECHNOLOGY INITIATIVE

# Video Meeting System

Implementation-specific BRD, architecture, deployment blueprint and developer handover

**Prarambh Portal integration • PLVC workflow • Automatic recording**

On-premises hosting | Windows Server 2019 + Ubuntu | Microsoft SQL Server

30 OCT 2026

# Executive recommendation

### Video platform

LiveKit self-hosted + WebRTC SFU
Embedded TURN initially

### Recording

LiveKit Egress workers
Composite MP4 after consent

### Enterprise integration

Existing Prarambh authentication
Microsoft SQL Server + SMTP

## Design principle

Keep the existing Prarambh portal and SQL Server on Windows infrastructure. Run media, recording and object-storage services on dedicated Ubuntu servers inside the OneInsure premises.

No paid video-conferencing subscription is required. Budget remains necessary for compute, network capacity, storage, backups, certificates and operations.

# Confirmed environment and design assumptions

| Area | Confirmed choice | Implementation implication |
|---|---|---|
| Portal | ASP.NET MVC / ASP.NET Core | Existing portal remains system of record |
| Database | Microsoft SQL Server | Reuse existing employee/customer/policy masters |
| Hosting | Internal premises | Windows Server 2019 + new Ubuntu servers |
| Concurrency | 10–25 simultaneous meetings | Capacity target to be proven by load test |
| Recording storage | On-premises | Private S3-compatible object storage / MinIO |
| Email | Existing company SMTP | Invitations, reminders, failure alerts |
| Customer access | Browser link | No customer account or app installation |

**Assumption to validate: available public/reachable IP, DNS and firewall policy for customer access to the on-premises video endpoint.**

# Business scope and outcomes

### Employee workflow

Select customer and policy
Schedule / reschedule / cancel
Join and admit customer
Access own recordings

### Customer workflow

Open secure invitation
Review recording notice
Give consent
Wait for employee admission

### Admin / compliance

Monitor meetings and recordings
View authorized employee activity
Audit playback / download
Track storage and failures

## Initial release excludes

- AI transcription, AI summaries, facial recognition and native mobile apps.
- Replacement of Prarambh authentication or customer/policy master data.
- Paid third-party conferencing services or SaaS recording storage.

# Roles, ownership and access rules

| Role | Permissions | Control |
|---|---|---|
| Employee / creator | Create and manage own meetings; access own recordings | Creator ID retained permanently |
| Customer | Join only invited meeting; provide consent; no account | No access to portal records |
| Team leader | Read permitted team meetings and recordings | Team scope enforced server-side |
| Admin | Central dashboard and authorized recording access | All actions audited |
| Compliance / audit | Read-only access within approved scope | No edit/delete permissions |
| Super admin | System configuration and retention controls | Privileged access monitored |

**Never rely on frontend visibility alone. Enforce every permission in backend APIs and signed playback/download flows.**

# Functional requirements checklist

### Scheduling

Customer, policy, date/time, duration, purpose, remarks; timezone-aware validation

### Invitations

Secure random token, employee + customer email, reminders, revoke on cancellation

### Meeting

HD video/audio, camera/mic, screen share, chat, fullscreen, waiting room, reconnect

### Recording

Consent gate, automatic Egress start, stop on meeting end, file verification

### Recording library

Employee ownership, admin RBAC, playback, download, audit

### Operations

Meeting statistics, storage metrics, failure queue, alerts, health dashboard

# End-to-end PLVC meeting lifecycle

**1 Schedule**
Employee selects customer + policy

**2 Invite**
Secure link + email delivery

**3 Join**
Employee joins; customer opens browser link

**4 Consent**
Customer and required participants consent

**5 Admit**
Employee admits customer from waiting room

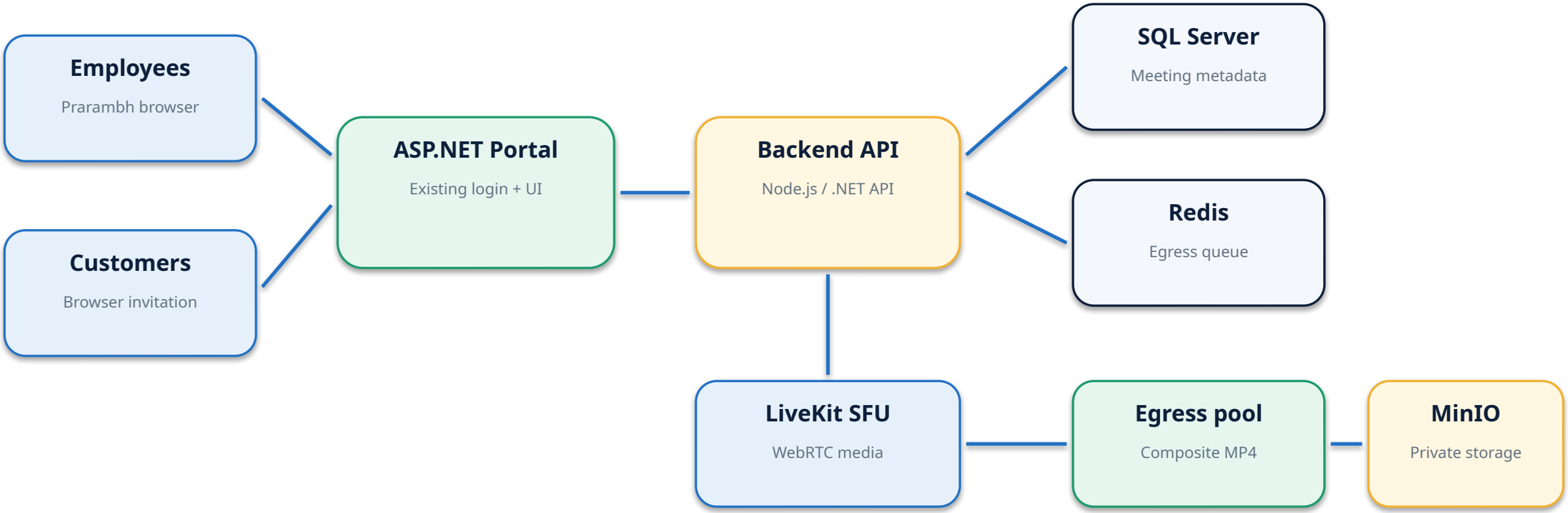
**6 Record**
Backend starts Egress automatically

**7 Complete**
Meeting ends; Egress stops

**8 Verify**
MP4 validated, checksum stored, status completed

**Formal PLVC completion requires verified recording availability unless an approved, audited exception path is used.**

# Target on-premises architecture




Internal service traffic stays on the premises network. Customer media access requires an externally reachable, secured LiveKit endpoint and approved firewall/NAT design.

# Network zones, DNS and port plan

| DNS / service | Protocol | Ports (example) | Purpose |
|---|---|---|---|
| meeting.oneinsure.in | HTTPS | 443/TCP | Customer join UI |
| api-meeting.oneinsure.in | HTTPS | 443/TCP | REST APIs; reverse proxy |
| livekit.oneinsure.in | WSS + WebRTC | 443/TCP; 7881/TCP; 7882/UDP or media UDP range | Signaling and media |
| TURN endpoint | TURN/TLS, TURN/UDP | 5349/TCP or 443/TCP; 3478/UDP | Relay when direct ICE fails |
| LiveKit media | ICE/UDP | 50000–60000/UDP (example range) | Media traffic if range mode selected |
| MinIO internal | S3 HTTPS | Internal only | Egress writes recordings |
| SQL / Redis | DB / cache | Internal only | Never internet-exposed |

**Final firewall rules must match the selected LiveKit config. Choose UDP mux OR a UDP port range deliberately; do not blindly open both.**

# Technology selection and rationale

| Component | Role | Reason |
|---|---|---|
| WebRTC + LiveKit | Media transport + SFU | Programmable embedded meetings; self-hosted |
| LiveKit Egress | Room composite recording | Separate worker pool; MP4/HLS outputs |
| ASP.NET MVC/Core | Portal UI / integration | Reuse current portal stack and sessions |
| Node.js / TypeScript | Meeting orchestration API | LiveKit server SDK and async workers |
| SQL Server | System of record | Employee/customer/policy/meeting relationships |
| Redis | Egress coordination / queue | Private network; persistent queue strategy |
| MinIO | On-prem object storage | S3-compatible Egress destination |
| SMTP | Email invitations | Reuse existing mail relay |
| Prometheus/Grafana | Monitoring | Media, egress, storage and API metrics |

# SQL Server data model

### VM_Meeting

EmployeeID
CustomerID
PolicyNumber
RoomName
Status

### VM_Participant

MeetingID
Type / Identity
Join / Leave

### VM_Invitation

MeetingID
TokenHash
Expiry / Revoked

### VM_Consent

ParticipantID
Consent version
Decision / time

### VM_Recording

MeetingID
EmployeeID
EgressID
ObjectKey / hash

### VM_RecordingJob

EgressID
Retry / error
Job state

### VM_AuditLog

Actor / action
IP / timestamp

### VM_Notification

Recipient
Template / status

Core relationship: VM_Meeting is the aggregate root; recording ownership is copied from the immutable meeting creator, not the employee currently viewing the record.

# SQL migration – core meeting table

```
CREATE TABLE dbo.VM_Meeting (
  MeetingID BIGINT IDENTITY(1,1) PRIMARY KEY,
  MeetingCode VARCHAR(30) NOT NULL UNIQUE,
  EmployeeID BIGINT NOT NULL,
  EmployeeCode VARCHAR(30) NOT NULL,
  EmployeeName NVARCHAR(150) NOT NULL,
  CustomerID BIGINT NOT NULL,
  CustomerName NVARCHAR(150) NOT NULL,
  PolicyID BIGINT NULL,
  PolicyNumber VARCHAR(100) NULL,
  LiveKitRoomName VARCHAR(100) NOT NULL UNIQUE,
  MeetingPurpose VARCHAR(50) NOT NULL,
  ScheduledStart DATETIME2 NOT NULL,
  ScheduledEnd DATETIME2 NOT NULL,
  ActualStart DATETIME2 NULL, ActualEnd DATETIME2 NULL,
  MeetingStatus VARCHAR(30) NOT NULL DEFAULT 'SCHEDULED',
  RecordingRequired BIT NOT NULL DEFAULT 1,
  CreatedBy BIGINT NOT NULL,
  CreatedDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);
```

### Migration rule

Inspect existing Prarambh master keys and data types before adding FKs. Use actual EmployeeID / CustomerID / PolicyID references.

### Time handling

Store UTC in DATETIME2. Convert to Asia/Kolkata only for UI and email. Validate daylight/timezone inputs.

# SQL migration – supporting tables and indexes

```
-- Consent and recording metadata (core columns)
CREATE TABLE dbo.VM_Consent (
  ConsentID BIGINT IDENTITY PRIMARY KEY,
  MeetingID BIGINT NOT NULL,
  ParticipantID BIGINT NOT NULL,
  ConsentType VARCHAR(30) NOT NULL,
  ConsentVersion VARCHAR(30) NOT NULL,
  ConsentStatus VARCHAR(20) NOT NULL,
  ConsentDate DATETIME2 NULL,
  NoticeVersion VARCHAR(30) NOT NULL
);

CREATE TABLE dbo.VM_Recording (
  RecordingID BIGINT IDENTITY PRIMARY KEY,
  MeetingID BIGINT NOT NULL,
  EmployeeID BIGINT NOT NULL,
  CustomerID BIGINT NOT NULL,
  EgressID VARCHAR(100) NULL,
  StorageBucket VARCHAR(100) NULL,
  StorageObjectKey NVARCHAR(1000) NULL,
  FileSizeBytes BIGINT NULL,
  FileChecksum VARCHAR(128) NULL,
  RecordingStatus VARCHAR(30) NOT NULL DEFAULT 'PENDING'
);

CREATE INDEX IX_VM_Meeting_Employee_Date
  ON dbo.VM_Meeting(EmployeeID, ScheduledStart);
CREATE INDEX IX_VM_Meeting_Status_Date
  ON dbo.VM_Meeting(MeetingStatus, ScheduledStart);
CREATE INDEX IX_VM_Recording_Employee
  ON dbo.VM_Recording(EmployeeID, RecordingStatus);
```

## Additional tables

VM_Participant • VM_Invitation • VM_RecordingJob • VM_AuditLog • VM_Notification

## Production hardening

Add FKs after master review; immutable consent/audit events; retention and legal-hold fields; row-level access tests.

# REST API contract – v1

| Verb | Endpoint | Purpose |
|---|---|---|
| POST | /meetings | Create meeting; server derives employee identity |
| GET | /meetings | List meetings within caller's scope |
| GET | /meetings/{id} | Meeting details and state |
| PUT | /meetings/{id} | Reschedule; regenerate/revoke invitation |
| POST | /meetings/{id}/cancel | Cancel and revoke access |
| POST | /meetings/{id}/join-token | Issue short-lived LiveKit JWT |
| POST | /meetings/{id}/consent | Record participant consent |
| POST | /meetings/{id}/admit | Employee admits waiting participant |
| POST | /meetings/{id}/end | End meeting; trigger recording stop |
| GET | /recordings | Employee-scoped recording library |
| POST | /recordings/{id}/playback | Return short-lived signed URL |
| POST | /webhooks/livekit | Verified LiveKit events |
| POST | /webhooks/egress | Verified Egress status events |

# Backend source structure

```
oneinsure-video-api/
  src/
    app.ts
    config/          env, database, livekit, storage
    middleware/      auth, rbac, csrf, rateLimit, errors
    modules/
      meetings/      controller, service, repository, dto
      participants/  admission, join-token
      consent/       consent.service, notice versions
      recordings/    egress.service, verifier, playback
      invitations/   token.service, email templates
      notifications/ smtp, reminder worker
      webhooks/      livekit, egress signature handlers
      audit/         audit.service
      admin/         reports, storage metrics
    jobs/            reminder, recording-reconcile, retry
    shared/          logger, crypto, validation
    migrations/
    tests/
  Dockerfile
  package.json
  .env.example
```

## Application boundary

Portal calls backend. Backend owns tokens, meeting state, consent, Egress, storage credentials and authorization.

## Workers

Separate queue consumers handle SMTP reminders, Egress completion, file verification and retries.

## Runtime

TypeScript + Express/NestJS; SQL Server driver; LiveKit server SDK; structured logs.

# Frontend and Prarambh module structure

```
Prarambh (existing ASP.NET MVC/Core)
  Areas/VideoMeetings/
    Controllers/
      MeetingsController
      RecordingsController
      AdminMeetingsController
    Views/
      Meetings/Index.cshtml
      Meetings/Create.cshtml
      Meetings/Details.cshtml
      Meetings/Join.cshtml
      Recordings/Index.cshtml
      Admin/Dashboard.cshtml
    wwwroot/js/video/
      meeting-scheduler.ts
      meeting-room.ts
      customer-join.ts
      recording-library.ts
    wwwroot/css/video/
      meeting.css

Optional separate SPA:
  video-web/
    src/pages/
    src/components/
    src/lib/livekit/
    src/api/
    src/auth/
```

### Recommended integration

Embed a Video Meetings area into existing Prarambh navigation and reuse the existing server-side employee session.

### Customer join surface

Host a minimal responsive join page on meeting.oneinsure.in. It must not expose portal session or customer master data.

### Browser SDK

Use LiveKit Client SDK; camera/mic permissions require HTTPS and explicit browser permission.

# LiveKit configuration – production baseline

```
port: 7880
log_level: info

rtc:
  tcp_port: 7881
  # Select ONE media mode:
  # A) UDP range:
  port_range_start: 50000
  port_range_end: 60000
  use_external_ip: true

redis:
  address: redis.internal:6379
  username: livekit
  password: ${REDIS_PASSWORD}

keys:
  ONEINSURE_API_KEY: ${LIVEKIT_API_SECRET}

turn:
  enabled: true
  domain: turn.oneinsure.in
  tls_port: 5349
  udp_port: 3478

prometheus_port: 6789
```

### Security

Use a trusted TLS certificate. Keep API secret in secret manager / protected env file. Never commit config secrets.

### Network topology

The sample is illustrative. For production, follow LiveKit's current VM/Compose deployment guidance and network mode requirements.

### TURN

Embedded TURN can provide TURN/UDP and TURN/TLS. Validate NAT, public IP and firewall behavior.

# Docker Compose – service layout (illustrative)

```yaml
services:
  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes: [redis_data:/data]
    networks: [private]

  livekit:
    image: livekit/livekit-server:<PINNED_VERSION>
    command: ["--config", "/etc/livekit.yaml"]
    volumes: ["./livekit.yaml:/etc/livekit.yaml:ro"]
    depends_on: [redis]
    network_mode: host
    restart: unless-stopped

  egress:
    image: livekit/egress:<PINNED_VERSION>
    environment:
      EGRESS_CONFIG_FILE: /etc/egress.yaml
    volumes: ["./egress.yaml:/etc/egress.yaml:ro"]
    depends_on: [redis]
    # Dedicated Linux worker; validate required shared memory
    shm_size: "2gb"
    restart: unless-stopped

  api:
    image: registry.local/oneinsure/video-api:<VERSION>
    env_file: .env.production
    ports: ["127.0.0.1:3000:3000"]
    networks: [private]
    restart: unless-stopped

networks:
  private:
    internal: true
volumes:
  redis_data:
```

## Deployment notes

Use pinned image versions, health checks, secret files, resource limits, log rotation and tested upgrades.

## Important topology

Host networking is commonly recommended for LiveKit media on Linux. This example is a service layout, not a copy/paste final production compose.

# Egress and MinIO configuration

```
# egress.yaml – illustrative
api_key: ONEINSURE_API_KEY
api_secret: ${LIVEKIT_API_SECRET}
ws_url: wss://livekit.oneinsure.in

redis:
  address: redis.internal:6379
  username: egress
  password: ${REDIS_EGRESS_PASSWORD}

# S3-compatible destination is supplied
# in the StartEgress request:
# endpoint: https://minio.internal:9000
# bucket: oneinsure-recordings
# force_path_style: true
```

## Egress worker sizing

LiveKit recommends at least 4 CPU and 4 GB RAM per Egress instance. Composite room recording can need more; benchmark before sizing worker count.

## MinIO bucket policy

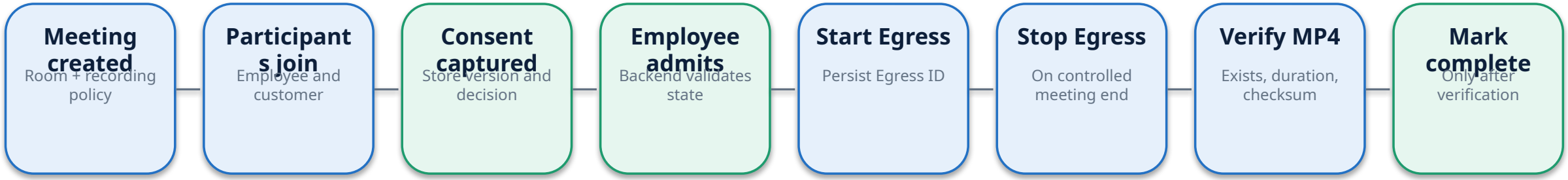
Private bucket; Egress write-only credentials; API read via short-lived presigned URLs; versioning/replication as approved.

## Object key pattern

recordings/YYYY/MM/DD/{opaque-employee-key}/{meeting-code}.mp4

Avoid customer name / policy number in object path.

# Automatic recording – consent-gated orchestration




## Recording failure policy

If consent is missing or Egress start fails, do not mark PLVC complete. Notify employee; open admin incident; allow only audited exception workflow.

## Reliability controls

Durable job table • idempotency key • webhook signature verification • retry queue • periodic reconciliation • checksum.

# API example – create meeting and issue join token

```
POST /api/v1/meetings
Authorization: existing Prarambh session / SSO
Content-Type: application/json

{
  "customerId": 50025,
  "policyNumber": "POL123456",
  "scheduledStart": "2026-10-30T11:30:00+05:30",
  "durationMinutes": 25,
  "purpose": "PLVC"
}

POST /api/v1/meetings/{id}/join-token
Backend checks:
- caller identity and role
- meeting state and invitation validity
- participant scope / consent state
- grant is least-privilege
Response:
{
  "serverUrl": "wss://livekit.oneinsure.in",
  "token": "<short-lived-JWT>"
}
```

## Identity rule

Employee ID comes from authenticated Prarambh server context. Never accept employeeId / isEmployee from untrusted request JSON.

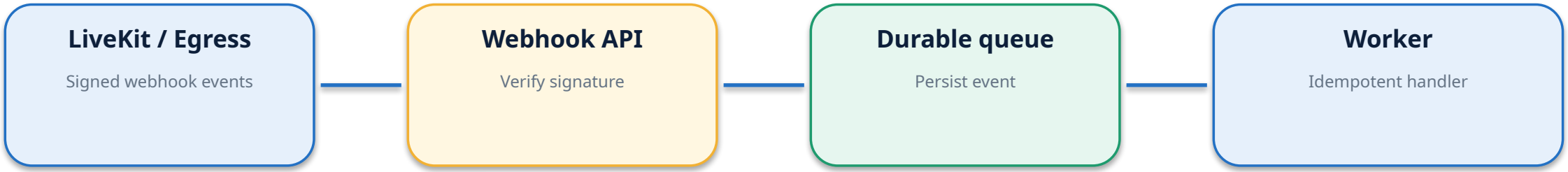
## Token rule

Short TTL; room-scoped; opaque participant identity; customer cannot receive roomAdmin; protect join-token endpoint with rate limits.

## Meeting code

Human-readable OI-VM code is for support only; access relies on high-entropy invitation token.

# Webhook, queue and recording recovery




### On Egress complete

Check job status; locate object; validate non-zero size; inspect MP4; calculate SHA-256; update SQL transactionally.

### On transient failure

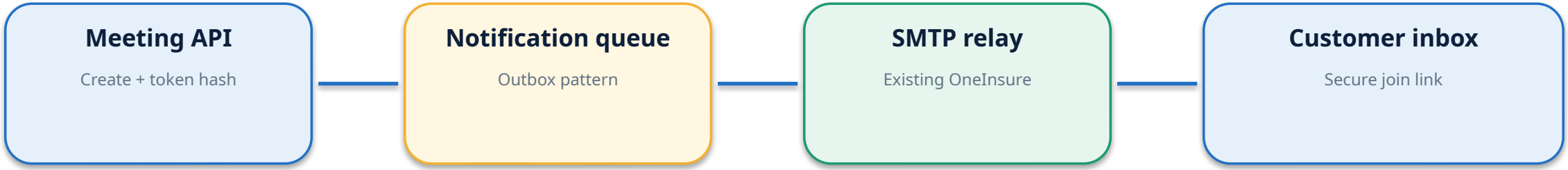
Retry with backoff; retain local spool until confirmed upload; alert on retry exhaustion.

### Reconciliation

Scheduled job finds stale STARTING / PROCESSING jobs and compares Egress state + object storage.

**Do not rely on a single webhook delivery or in-memory process. Persist state transitions and make every handler safe to replay.**

# Customer invitation, SMTP and reminder workflow




| Trigger | Email |
| --- | --- |
| At creation | Invitation |
| 24 hours before | Reminder |
| 1 hour before | Reminder |
| 10 minutes before | Final reminder |
| Reschedule / cancel | Updated / cancellation email |
| Recording complete | Employee notification |

Use an outbox table + background worker so email outages do not roll back a valid meeting record.

# Prarambh integration steps

- Inspect portal framework version, authentication/session model and employee master key.
- Expose server-side integration service; never put LiveKit API secret in browser code.
- Add Video Meetings navigation, employee dashboard and schedule form.
- Resolve customer and policy from existing master APIs / stored procedures.
- Call meeting backend using authenticated service-to-service identity.
- Embed employee meeting room; host customer join page separately.
- Apply employee/team/admin authorization on every recording query.
- Add audit events, monitoring and approved retention controls.

**Critical integration rule**

Do not create a

# Security, privacy and compliance controls

| Control area | Required implementation |
|---|---|
| Transport | HTTPS/WSS; WebRTC DTLS-SRTP; trusted certificates |
| At rest | SQL encryption where licensed/configured; encrypted disk/object storage; protected backups |
| Access | RBAC; employee ownership; team scope; least privilege; MFA for privileged roles |
| Invitations | Random high-entropy token; hash at rest; expiry; revoke; rate-limit |
| Recording | Consent evidence; visible recording indicator; server-side Egress; access audit |
| Storage | Private bucket; short-lived presigned URLs; no anonymous listing |
| Webhooks | Signature validation; replay/idempotency protection |
| Privacy | Approved notice, purpose limitation, retention, legal hold, access and deletion workflow |

**Legal/compliance must approve lawful basis, notice wording, retention and withdrawal handling before pilot.**

# Capacity planning for 10–25 concurrent meetings

| Service | Initial sizing | RAM | Notes |
|---|---|---|---|
| LiveKit SFU | 8–16 vCPU | 16–32 GB | Dedicated media node; high network throughput |
| Egress workers | 4+ vCPU / worker | 4+ GB / worker | Benchmark composite recordings; scale worker pool |
| Backend API | 4 vCPU | 8 GB | API + background workers separated at scale |
| Redis | 2–4 vCPU | 4–8 GB | Private; persistence/HA as needed |
| MinIO | 4–8 vCPU | 16 GB | Disk throughput and capacity based on retention |
| Monitoring | 2–4 vCPU | 8 GB | Metrics, logs, alerting |

### Media capacity

Concurrency depends on resolution, bitrate, packet loss, TURN ratio and room topology.

### Recording capacity

Room-composite Egress is CPU-intensive; size by measured jobs per worker, not meeting count alone.

# On-prem recording storage estimate

## Planning example

1,000 meetings / month
25 minutes average
1.5 Mbps composite bitrate

**≈ 281 GB per month**
**≈ 3.38 TB for 12 months**

### Formula

GB ≈ meetings × minutes ÷ 60 × Mbps × 3600 ÷ 8 ÷ 1000

### Not included

Backup copies, replication, filesystem/object overhead, higher bitrate, failed/retry artifacts and retention/legal holds.

**Measure actual MP4 size from a pilot. Set storage alerts at 70%, 80% and 90%; test restore and lifecycle rules.**

# Operations, monitoring and recovery

| Domain | Monitor / alert |
| --- | --- |
| Media | Active rooms, participants, packet loss, RTT, jitter, TURN allocations |
| Egress | Worker up/down, queue depth, jobs running, failures, processing duration |
| Storage | Capacity, object count, upload latency, checksum mismatch, backup status |
| Backend | API latency, error rate, auth failures, queue lag, SMTP retries |
| Database | Connections, query latency, deadlocks, backup age |
| Security | Denied access, token abuse, playback/download audit, webhook failures |

**Incident response**

Recording failure creates ticket/alert; preserve diagnostic metadata; retry; inform employee/admin.

**Recovery**

Durable jobs, database backups, replicated storage, tested restore and documented failover.

# Testing and acceptance criteria

| Test area | Acceptance coverage |
|---|---|
| Functional | Schedule, invite, join, waiting room, admit, camera, mic, screen share, chat |
| Consent | Consent granted/denied; no recording before required consent |
| Recording | Start automatically; stop on end; valid MP4; correct mapping; checksum |
| Authorization | Employee cannot see another employee's recordings; admin scope enforced |
| Network | Office, home Wi-Fi, mobile data, VPN/firewall, TURN fallback |
| Load | 5 / 10 / 25 concurrent rooms and corresponding recording load |
| Recovery | Egress crash, webhook replay, SQL outage, SMTP outage, storage full |
| Browser | Chrome, Edge, Firefox, Safari, Android Chrome, iOS Safari |

**Pilot gate: zero untracked recordings; no unauthorized recording access; measured 25-room target with acceptable quality and recovery.**

# Implementation roadmap and delivery gates

### W1 | Infra & network

DNS, TLS, Docker, LiveKit, Redis, TURN

### W2–3 | Backend & data

SQL migrations, APIs, auth, invitations

### W4–5 | Meeting UI

Employee/customer browser, waiting room, controls

### W6–7 | Recording

Egress, consent, MinIO, verification, retry

### W8 | Prarambh

Portal menu, master integration, RBAC

### W9 | Admin

Reports, storage, audit, monitoring

### W10–11 | QA / load

Security, browser, 25-room capacity

### W12–13 | Pilot / rollout

UAT, training, production handover

Indicative 11–13 weeks, subject to team size, Prarambh API access, network approvals, procurement and compliance sign-off.

# Key risks and decisions before build

| Risk / decision | Required action |
|---|---|
| On-prem customer reachability | Public/reachable endpoint, DNS, firewall, NAT and ISP approval required |
| Recording consent | Legal/compliance approves notice, evidence and exception handling |
| Capacity | Benchmark 10–25 media rooms plus Egress jobs at target bitrate |
| Storage retention | Approve duration, backup copies, legal hold and deletion workflow |
| Portal authentication | Confirm exact MVC/Core version and session/SSO model |
| Network design | Choose UDP range or UDP mux; TURN/TLS fallback; test corporate firewalls |
| Licensing | Review versions/licences of LiveKit, Egress, MinIO and dependent components |

**Do not call the system production-ready until the above decisions are closed and acceptance tests pass.**

# Official technical references

| | |
|---|---|
| **LiveKit self-hosting** | https://docs.livekit.io/transport/self-hosting/ |
| **LiveKit deployment** | https://docs.livekit.io/transport/self-hosting/deployment/ |
| **LiveKit ports & firewall** | https://docs.livekit.io/transport/self-hosting/ports-firewall/ |
| **LiveKit Egress self-hosting** | https://docs.livekit.io/transport/self-hosting/egress/ |
| **Egress composite recording** | https://docs.livekit.io/transport/media/ingress-egress/egress/composite-recording/ |
| **Egress auto recording** | https://docs.livekit.io/transport/media/ingress-egress/egress/autoegress/ |
| **Egress S3-compatible output** | https://docs.livekit.io/transport/media/ingress-egress/egress/outputs/ |
| **LiveKit JS server SDK** | https://github.com/livekit/server-sdk-js |
| **Docker Compose production** | https://docs.docker.com/compose/how-tos/production/ |
| **WebRTC overview** | https://webrtc.org/ |

*Reference links should be rechecked against the pinned software versions during implementation.*

# A secure, self-hosted PLVC video and recording platform

Integrated with Prarambh. Owned by OneInsure. Built around auditable consent, reliable recording and employee-wise access.

### Prove first

Two-way video + network paths

### Then prove

Consent-gated auto recording

### Then scale

25-room load + recovery tests